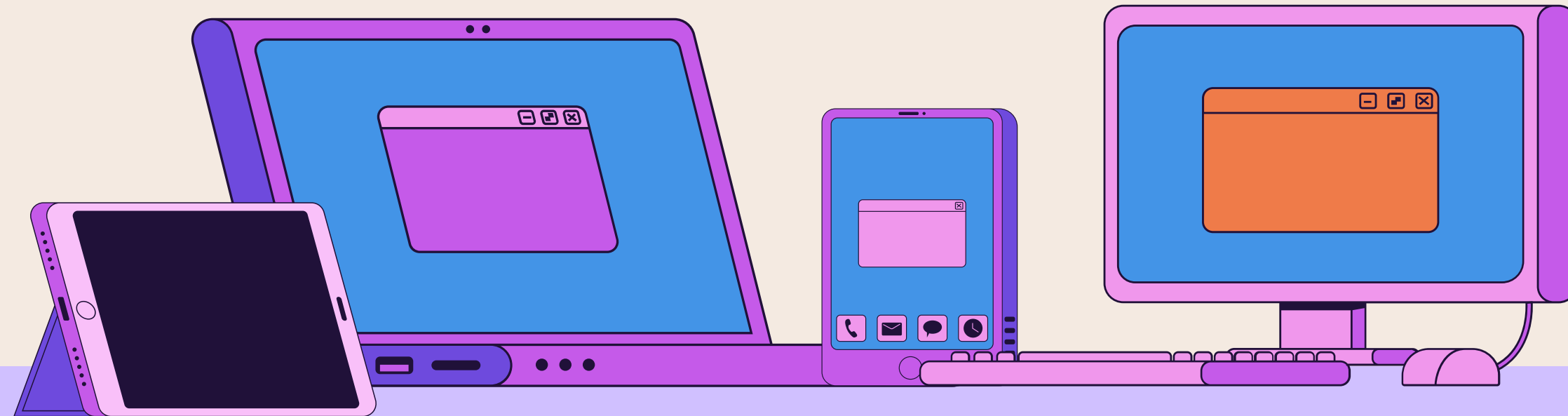


oleh Kelompok D

PERANCANGAN DAN IMPLEMENTASI BASIS DATA RELASIONAL PADA SISTEM RESERVASI HOTEL DENGAN MENGUNAKAN MYSQL



LATAR BELAKANG

- Industri perhotelan membutuhkan sistem informasi yang cepat, akurat, dan efisien.
- Pengelolaan data secara manual berisiko menyebabkan double booking, kesalahan data, dan keterlambatan layanan.
- Basis data relasional digunakan untuk mengelola data hotel secara terstruktur dan terintegrasi.
- Oleh karena itu, dirancang sistem basis data reservasi hotel yang mencakup proses reservasi, check-in, check-out, pembayaran, dan pengelolaan data pendukung lainnya.

RUMUSAN MASALAH

- 01 Bagaimana merancang ERD dan skema relasional sistem reservasi hotel?
- 02 Bagaimana mengimplementasikan basis data hingga memenuhi normalisasi 3NF?
- 03 Bagaimana membuat query dan objek basis data pendukung operasional hotel?



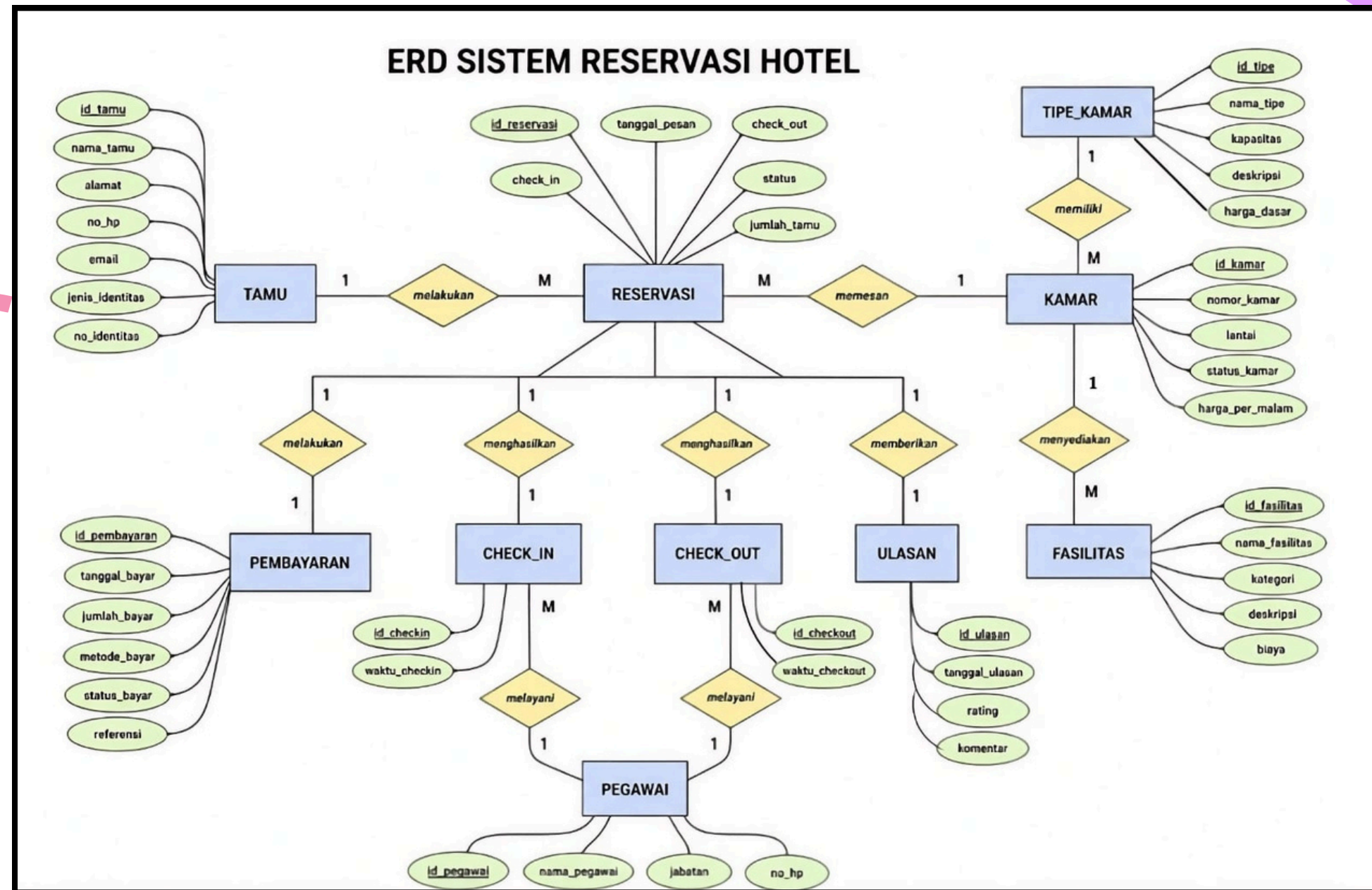
IDENTIFIKASI PENGGUNA

Pengguna	Peran Utama	Hak Akses Utama
Manajer	Monitoring dan evaluasi operasional hotel	Melihat reservasi, pembayaran, ulasan, dan laporan
Admin	Mengelola data dan sistem	Mengelola data pegawai, kamar, tipe kamar, dan fasilitas
Resepsionis	Melayani transaksi tamu	Mengelola tamu, reservasi, check-in, check-out, pembayaran
Tamu	Menggunakan layanan hotel	Registrasi, reservasi kamar, melihat status reservasi, memberi ulasan

Kebutuhan Fungsional Sistem

Kategori	Fungsi Utama
Manajemen Data	Mengelola data tamu, pegawai, kamar, tipe kamar, fasilitas, dan ulasan
Transaksi Hotel	Mengelola reservasi, check-in, check-out, dan pembayaran
Informasi Kamar	Menampilkan ketersediaan kamar dan memperbarui status kamar otomatis
Pengolahan Data	Menghitung biaya menginap dan menampilkan riwayat transaksi tamu
Laporan & Pencarian	Menyediakan pencarian data dan laporan operasional hotel
Integritas Data	Menjaga konsistensi hubungan antar data dalam sistem

ENTITY RELATIONSHIP DIAGRAM (ERD)

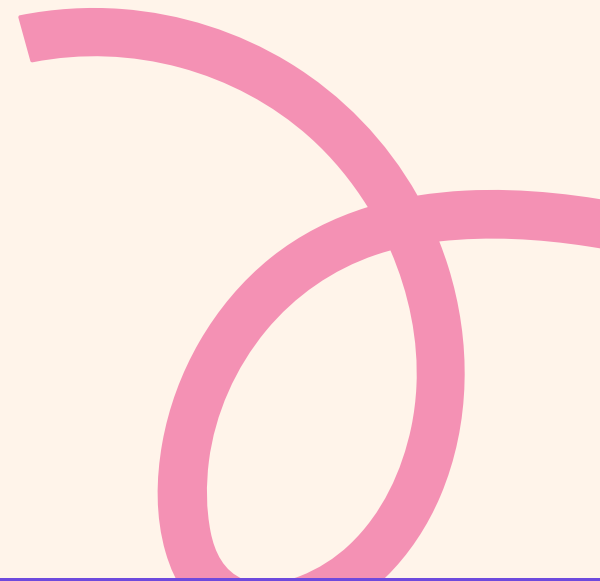


BUSINESS RULES

1. Kamar tidak bisa dipakai dua tamu sekaligus. Kamar yang sedang ditempati tamu tidak boleh dipesan atau di-check-in oleh tamu lain, sampai tamu sebelumnya sudah checkout.
2. Tanggal check-in harus sebelum check-out. Tanggal check-in wajib lebih awal dari tanggal check-out, dengan minimal menginap 1 malam. Reservasi tidak bisa dibuat untuk hari yang sudah lewat.
3. Check-in hanya boleh jika sudah bayar. Tamu hanya bisa melakukan check-in jika pembayaran sudah lunas (atau sudah membayar uang muka sesuai kebijakan hotel). Reservasi yang belum dibayar tidak boleh diproses check-in.
- 4.



PROSES NORMALISASI



Tahap	Hasil
0NF	Seluruh data masih dalam satu tabel besar dan terjadi redundansi
1NF	Data sudah atomik dan tidak terdapat grup berulang
2NF	Data dipisahkan menjadi beberapa tabel untuk menghilangkan ketergantungan parsial
3NF	Ketergantungan transitif dihilangkan sehingga setiap atribut bergantung pada primary key

- Setiap tabel memiliki primary key.
- Hubungan antar tabel menggunakan foreign key.
- Struktur telah memenuhi bentuk normal ketiga (3NF).



TABEL HASIL
NORMALISASI 3NF

No	Nama Tabel
1	TAMU
2	TIPE_KAMAR
3	KAMAR
4	RESERVASI
5	PEMBAYARAN
6	CHECK_IN
7	CHECK_OUT
8	PEGAWAI
9	FASILITAS
10	ULASAN

DDL (Data Definition Language)

-- 3. TABEL TAMU

```
CREATE TABLE tamu (  
  id_tamu VARCHAR(7) NOT NULL,  
  nama_tamu VARCHAR(100) NOT NULL,  
  alamat VARCHAR(200) NULL,  
  no_hp VARCHAR(20) NOT NULL,  
  email VARCHAR(100) NULL,  
  jenis_identitas ENUM('KTP','SIM','Passport') NOT NULL DEFAULT 'KTP',  
  no_identitas VARCHAR(20) NOT NULL,  
  CONSTRAINT pk_tamu PRIMARY KEY (id_tamu),  
  CONSTRAINT uq_tamu_identitas UNIQUE (jenis_identitas,  
  no_identitas),  
  CONSTRAINT uq_tamu_email UNIQUE (email) );
```

```
CREATE INDEX idx_tamu_nama ON tamu(nama_tamu);
```

- Jenis identitas hanya boleh:
 - KTP
 - SIM
 - Passport
 - Jika tidak diisi, otomatis bernilai KTP.
- Membuat indeks pada kolom nama_tamu., untuk Mempercepat pencarian data berdasarkan nama tamu.

DML (Data Manipulation Language)

-- TAMU (20 baris)

```
INSERT INTO tamu (id_tamu, nama_tamu, alamat, no_hp, email,  
jenis_identitas, no_identitas)  
VALUES  
( 'TM001', 'Budi Santoso', 'Jakarta', '081234567801', 'budi@gmail.com', 'KTP',  
'3171111111111110'),
```

Kolom	Kondisi
id_tamu	Unik (TM001-TM020)
nama_tamu	Terisi semua
alamat	Terisi semua
no_hp	Unik dan tidak kosong
email	Unik dan tidak kosong
jenis_identitas	19 KTP, 1 SIM
no_identitas	Unik semua

QUERY SEDERHANA

-- QUERY 1 (Sederhana) - Daftar semua kamar beserta tipe kamarnya

```
SELECT
  k.id_kamar,
  k.nomor_kamar,
  k.lantai,
  k.status_kamar,
  t.nama_tipe,
  t.harga_dasar
FROM kamar k
JOIN tipe_kamar t ON k.id_tipe = t.id_tipe
ORDER BY k.lantai, k.nomor_kamar;
```

OUTPUT

	id_kamar	nomor_kamar	lantai	status_kamar	nama_tipe	harga_dasar
▶	K001	101	1	Terisi	Standard	350000.00
	K002	102	1	Terisi	Standard	350000.00
	K003	103	1	Dipesan	Standard	350000.00
	K045	104	1	Terisi	Standard	350000.00
	K052	105	1	Terisi	Standard	350000.00
	K059	106	1	Dipesan	Standard	350000.00
	K024	123	1	Terisi	Standard	350000.00
	K004	201	2	Terisi	Deluxe	550000.00
	K005	202	2	Terisi	Deluxe	550000.00
	K006	203	2	Dipesan	Deluxe	550000.00
	K038	222	2	Terisi	Deluxe	550000.00
	K007	301	3	Terisi	Superior	700000.00
	K008	302	3	Dipesan	Superior	700000.00

QUERY JOIN

-- QUERY 5 (JOIN ≥ 3 tabel) - Riwayat pembayaran lengkap dengan info tamu

-- Gabung: pembayaran + reservasi + tamu + kamar

```
SELECT
  pb.id_pembayaran,
  tm.nama_tamu,
  k.nomor_kamar,
  r.check_in,
  r.check_out,
  pb.jumlah_bayar,
  pb.metode_bayar,
  pb.status_bayar,
  pb.tanggal_bayar
FROM pembayaran pb
JOIN reservasi r ON pb.id_reservasi = r.id_reservasi
JOIN tamu tm ON r.id_tamu = tm.id_tamu
JOIN kamar k ON r.id_kamar = k.id_kamar
ORDER BY pb.tanggal_bayar;
```

OUTPUT

	id_pembayaran	nama_tamu	nomor_kamar	check_in	check_out	jumlah_bayar	metode_bayar	sta
▶	PB001	Budi Santoso	101	2026-06-05	2026-06-07	700000.00	Transfer	Lun
	PB002	Siti Aminah	201	2026-06-06	2026-06-08	1100000.00	QRIS	Lun
	PB003	Andi Wijaya	301	2026-06-10	2026-06-12	1400000.00	Transfer	Lun
	PB004	Rina Putri	102	2026-06-11	2026-06-13	700000.00	Cash	Lun
	PB005	Dimas Pratama	401	2026-06-15	2026-06-17	1900000.00	Transfer	Lun
	PB006	Dewi Lestari	202	2026-06-20	2026-06-22	1100000.00	QRIS	Lun
	PB007	Fajar Nugroho	203	2026-06-25	2026-06-27	550000.00	Transfer	Lun
	PB008	Maya Sari	302	2026-06-28	2026-06-30	700000.00	Transfer	Lun
	PB009	Rizki Akbar	103	2026-07-01	2026-07-03	350000.00	Cash	Lun
	PB010	Nanda Putri	501	2026-07-05	2026-07-07	1500000.00	Transfer	Lun
	PB011	Budi Santoso	509	2026-07-08	2026-07-10	700000.00	Transfer	Lun
	PB012	Siti Aminah	123	2026-07-11	2026-07-13	1100000.00	QRIS	Lun

TAMU DI ATAS RATA-RATA PEMBAYARAN (SUBQUERY)

```
-- QUERY 8 (Subquery) - Tamu yang total pembayarannya
-- melebihi rata-rata pembayaran seluruh tamu
SELECT
    tm.nama_tamu,
    tm.email,
    SUM(pb.jumlah_bayar) AS total_pembayaran
FROM tamu tm
JOIN reservasi r ON tm.id_tamu = r.id_tamu
JOIN pembayaran pb ON r.id_reservasi = pb.id_reservasi
GROUP BY tm.id_tamu, tm.nama_tamu, tm.email -- Mengelompokkan
data berdasarkan tamu
HAVING SUM(pb.jumlah_bayar) > ( -- Memfilter hasil setelah agregasi,
dan menampilkan data dengan total pembayaran > rata-rata
    SELECT AVG(sub.total)
    FROM (
        SELECT SUM(pb2.jumlah_bayar) AS total
        FROM pembayaran pb2
        JOIN reservasi r2 ON pb2.id_reservasi = r2.id_reservasi
        GROUP BY r2.id_tamu
    ) sub
)
ORDER BY total_pembayaran DESC;
```

OUTPUT

nama_tamu	email	total_pembayaran
Nanda Putri	nanda@gmail.com	4500000.00
Dimas Pratama	dimas@gmail.com	3800000.00
Andi Wijaya	andi@gmail.com	2800000.00

(Agregat + GROUP BY + HAVING)

```
-- QUERY 10 (Agregat + GROUP BY + HAVING) -
Pegawai yang menangani
-- lebih dari 3 proses check-in, beserta total transaksi
SELECT
  pg.id_pegawai,
  pg.nama_pegawai,
  pg.jabatan,
  COUNT(ci.id_checkin) AS jumlah_checkin,
  COUNT(co.id_checkout) AS jumlah_checkout,
  COUNT(ci.id_checkin) + COUNT(co.id_checkout) AS
total_transaksi
FROM pegawai pg
LEFT JOIN check_in ci ON pg.id_pegawai =
ci.id_pegawai
LEFT JOIN check_out co ON pg.id_pegawai =
co.id_pegawai
GROUP BY pg.id_pegawai, pg.nama_pegawai,
pg.jabatan
HAVING COUNT(ci.id_checkin) > 3
ORDER BY total_transaksi DESC;
```

- Query menampilkan pegawai yang menangani lebih dari 3 proses check-in.
- Hasil menunjukkan Ahmad Fauzi dan Yuni Astuti memiliki transaksi terbanyak (98 transaksi), sedangkan Dedi Saputra dan Rika Maharani masing-masing memiliki 18 transaksi.
- Data diurutkan dari total transaksi tertinggi ke terendah menggunakan ORDER BY.
- Output ini dapat digunakan untuk mengevaluasi kinerja dan produktivitas pegawai.

	id_pegawai	nama_pegawai	jabatan	jumlah_checkin	jumlah_checkout	total_transaksi
▶	P001	Ahmad Fauzi	Resepsionis	49	49	98
	P002	Yuni Astuti	Resepsionis	49	49	98
	P003	Dedi Saputra	Supervisor	9	9	18
	P004	Rika Maharani	Front Office	9	9	18



-- VIEW 2: v_statistik_kamar

-- Menampilkan statistik kinerja setiap tipe kamar: jumlah reservasi, total pendapatan, dan rata-rata rating ulasan.

-- Berguna bagi manajemen untuk evaluasi performa kamar.

```
CREATE VIEW v_statistik_kamar AS
SELECT
  t.id_tipe,
  t.nama_tipe,
  t.kapasitas,
  t.harga_dasar,
  COUNT(DISTINCT r.id_reservasi) AS total_reservasi,
  SUM(pb.jumlah_bayar) AS total_pendapatan,
  ROUND(AVG(pb.jumlah_bayar), 0) AS rata_pendapatan_per_res, --
  ROUND untuk membulatkan angka dari desimal ke int, misal 60000,75
  menjadi 60001
  ROUND(AVG(u.rating), 2) AS rata_rating,
  COUNT(DISTINCT u.id_ulasan) AS jumlah_ulasan -- DISTINT untuk
menghilangkan data yang duplikat sehingga hanya menampilkan nilai
unik
FROM tipe_kamar t
LEFT JOIN kamar k ON t.id_tipe = k.id_tipe
LEFT JOIN reservasi r ON k.id_kamar = r.id_kamar
LEFT JOIN pembayaran pb ON r.id_reservasi = pb.id_reservasi
LEFT JOIN ulasan u ON r.id_reservasi = u.id_reservasi
GROUP BY t.id_tipe, t.nama_tipe, t.kapasitas, t.harga_dasar;
-- Contoh penggunaan:
-- SELECT * FROM v_statistik_kamar ORDER BY total_pendapatan DESC;
```

VIEW

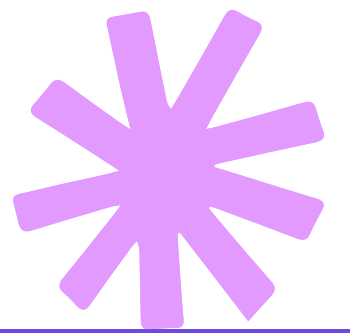
Tujuan :

- Menampilkan statistik setiap tipe kamar.
- Menghitung jumlah reservasi.
- Menghitung total pendapatan.
- Menghitung rata-rata pendapatan per reservasi.
- Menghitung rata-rata rating tamu.
- Menghitung jumlah ulasan.

Fungsi SQL	Kegunaan
COUNT()	Menghitung jumlah reservasi
SUM()	Menghitung total pendapatan
AVG()	Menghitung rata-rata
ROUND()	Membulatkan angka
DISTINCT	Menghilangkan data duplikat
LEFT JOIN	Menggabungkan tabel dan tetap menampilkan data meskipun belum ada transaksi

	id_tipe	nama_tipe	kapasitas	harga_dasar	total_reservasi	total_pendapatan	rata_pe
	T001	Standard	2	350000.00	7	6950000.00	992857
	T004	Family	4	950000.00	3	6300000.00	2100000
	T003	Superior	3	700000.00	3	3500000.00	1166667
	T002	Deluxe	2	550000.00	4	3450000.00	862500
	T005	Suite	4	1500000.00	3	2900000.00	966667

STORED PROCEDURE



```
-- STORED PROCEDURE
USE hotel_reservasi;
DELIMITER //
CREATE PROCEDURE sp_buat_reservasi(
    IN p_id_reservasi VARCHAR(5),
    IN p_id_tamu VARCHAR(7),
    IN p_id_kamar VARCHAR(5),
    IN p_check_in DATE,
    IN p_check_out DATE,
    IN p_jumlah_tamu TINYINT
)
BEGIN
    -- Insert reservasi baru
    INSERT INTO reservasi
    (id_reservasi, tanggal_pesan, check_in, check_out,
    jumlah_tamu, status_reservasi, id_tamu, id_kamar)
    VALUES
    (p_id_reservasi, CURDATE(), p_check_in, p_check_out,
    p_jumlah_tamu, 'Dipesan', p_id_tamu, p_id_kamar);
```

```
-- Update status kamar menjadi Dipesan
UPDATE kamar -- Mengupdate tabel
SET status_kamar = 'Dipesan' -- Mengubah status kamar setelah ada
yang reservasi
WHERE id_kamar = p_id_kamar; -- Hanya kamar yang dipilih
SELECT CONCAT('Reservasi ', p_id_reservasi, ' berhasil dibuat.') AS
hasil; -- CONCAT untuk menggabungkan teks (Reservasi R021 berhasil
dibuat)
END //
DELIMITER ;
CALL sp_buat_reservasi('R021','TM003','K045','2026-09-01','2026-09-
04',2);
```

OUTPUT

	hasil
▶	Reservasi R021 berhasil dibuat.

TRIGGER

```
-- TRIGGER 1: trg_after_checkout_update_kamar
-- Fungsi : Setelah baris baru masuk ke tabel check_out,
--   status kamar terkait otomatis diubah menjadi
--   'Tersedia', dan status reservasi diubah 'Selesai'.
--   Kejadian ini juga dicatat ke tabel audit_log.
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_after_checkout_update_kamar
AFTER INSERT ON check_out
FOR EACH ROW
BEGIN
```

```
  DECLARE v_id_kamar VARCHAR(5);
  DECLARE v_nama_tamu VARCHAR(100);
```

```
  -- Ambil id_kamar dari reservasi terkait
  SELECT r.id_kamar, tm.nama_tamu
  INTO v_id_kamar, v_nama_tamu
  FROM reservasi r
  JOIN tamu tm ON r.id_tamu = tm.id_tamu
  WHERE r.id_reservasi = NEW.id_reservasi;
```

```
  -- Update status kamar menjadi Tersedia
  UPDATE kamar
  SET status_kamar = 'Tersedia'
  WHERE id_kamar = v_id_kamar;
```

```
-- Update status reservasi menjadi Selesai
UPDATE reservasi
SET status_reservasi = 'Selesai'
WHERE id_reservasi = NEW.id_reservasi;
```

```
  -- Catat ke audit_log
  INSERT INTO audit_log (nama_tabel, operasi, id_record,
keterangan)
  VALUES (
    'check_out',
    'INSERT',
    NEW.id_checkout,
    CONCAT(
      'Check-out ', NEW.id_checkout,
      ' | Reservasi: ', NEW.id_reservasi,
      ' | Tamu: ', v_nama_tamu,
      ' | Kamar ', v_id_kamar, ' -> Tersedia',
      ' | Waktu: ', NEW.waktu_checkout
    )
  );
END//
DELIMITER ;
```


Function

-- FUNCTION : fn_hitung_total_tagihan

DELIMITER //

CREATE FUNCTION fn_hitung_total_tagihan(p_id_reservasi
VARCHAR(5))

RETURNS DECIMAL(12,2)

DETERMINISTIC

READS SQL DATA

BEGIN

DECLARE v_harga_dasar DECIMAL(12,2);

DECLARE v_lama INT;

DECLARE v_total DECIMAL(12,2);

SELECT

t.harga_dasar,

DATEDIFF(r.check_out, r.check_in)

INTO v_harga_dasar, v_lama

FROM reservasi r

JOIN kamar k ON r.id_kamar = k.id_kamar

JOIN tipe_kamar t ON k.id_tipe = t.id_tipe

WHERE r.id_reservasi = p_id_reservasi;

SET v_total = v_lama * v_harga_dasar;

RETURN v_total;

END//

DELIMITER ;

SELECT fn_hitung_total_tagihan('R001') AS total_tagihan;

Tujuan :

- Menghitung total biaya yang harus dibayar tamu.
- Berdasarkan lama menginap dan harga kamar.
- Dapat dipanggil kapan saja menggunakan ID reservasi.

Cara Kerja

ID Reservasi

↓

Ambil Harga Dasar Kamar

↓

Hitung Lama Menginap

(check_out - check_in)

↓

Total = Lama Menginap × Harga Dasar

↓

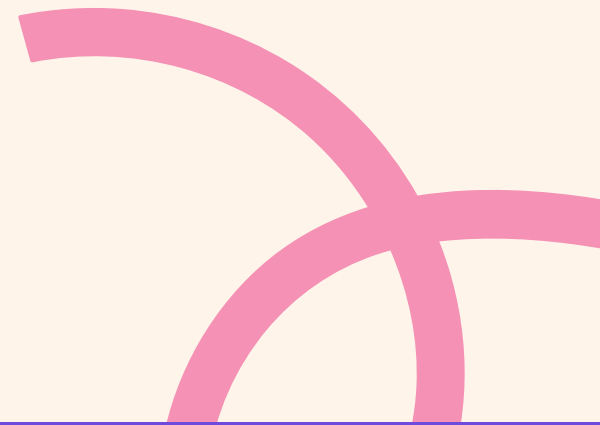
Tampilkan Total Tagihan

Output

Result Grid		Filter Rows:
	total_tagihan	
▶	700000.00	



PENUTUP



KESIMPULAN

- Berhasil merancang ERD dan skema relasional sistem reservasi hotel yang terdiri dari 10 entitas dan 10 tabel yang saling terhubung.
- Basis data berhasil diimplementasikan dan dinormalisasi hingga 3NF, serta dilengkapi berbagai constraint untuk menjaga integritas data.
- Berhasil mengembangkan query, view, stored procedure, trigger, dan function untuk mendukung otomatisasi dan efisiensi pengelolaan reservasi hotel.

Hasil: Sistem basis data reservasi hotel telah berjalan secara terintegrasi, konsisten, dan mendukung kebutuhan operasional hotel.

SARAN

- Menambahkan fitur autentikasi dan otorisasi pengguna untuk meningkatkan keamanan akses data.
- Mengintegrasikan basis data dengan aplikasi web atau mobile agar proses reservasi dapat dilakukan secara daring.
- Menerapkan mekanisme backup dan recovery secara berkala untuk menjaga keamanan data.
- Mengembangkan basis data dengan menambahkan fitur seperti promosi, paket menginap, room service, serta integrasi keuangan agar sistem lebih lengkap dan terintegrasi.

oleh Kelompok D

TERIMAKASIH ATAS PERHATIANNYA

